IMPLEMENTATION ASSIGNMENT 1

Student Record Management API

REST APIs using Spring Boot Framework: Build, Test Evidence and Report

Mahantesh R. Patted

USN 23BTRCT066 | B.Tech CSE

Jain (Deemed-to-be) University, Bengaluru

3 November 2026

Source code: GitHub repository `student-record-api`

Contents

1. Summary

2. Project Setup and Layered Design (Task 1)

3. REST Operations and Validation (Tasks 2 and 3)

4. Required Test Evidence

    4.1 Retrieve all students | 4.2 Valid creation | 4.3 Duplicate USN

    4.4 Invalid semester and blank fields | 4.5 Department filtering | 4.6 Update

    4.7 Deletion followed by a 404 lookup | 4.8 H2 query result

5. Startup Data and Logging (Task 4)

6. Build, Demonstration and Restart (Task 5)

7. Why the Service Layer Is Separate from the Controller

8. Submission Contents and Commands

1. Summary

A Spring Boot REST API that stores student records (generated id, unique USN, name, department, semester 1 to 8) in a file-based H2 database. All five assignment tasks are implemented and were demonstrated on the packaged executable JAR. An automated run of 20 scenarios passed with every expected status code, and every required piece of evidence below was captured live from the running application.

Item	Value
Java	OpenJDK 21.0.11 (LTS); project targets Java 17+
Framework	Spring Boot 3.5.0 (Spring Web, Spring Data JPA, Validation)
Build tool	Apache Maven 3.9.11
Database	H2 2.3.232, file-based: jdbc:h2:file:./data/studentdb
Artifact	target/student-api-0.0.1-SNAPSHOT.jar (executable JAR)
Test platform	Windows 11, run locally on port 8080
Result	20 / 20 automated scenarios passed; all required evidence captured

2. Project Setup and Layered Design (Task 1)

The project was generated with Spring Initializr (Spring Web, Spring Data JPA, Validation, H2) and is organised into separate layers. Every component receives its dependencies through its constructor; there is no field injection.

Layer / class	Responsibility
controller / StudentController	HTTP only: routes, @Valid request bodies, status codes (201, 200, 204)
service / StudentService	Business rules: USN normalisation and uniqueness, "PUT never creates", transactions, mutation logging
repository / StudentRepository	Spring Data JPA interface: existsByUsn, existsByUsnAndIdNot, findByDepartmentIgnoreCase
model / Student	JPA entity with an IDENTITY id and a unique constraint on usn
dto / StudentRequest, StudentResponse, ErrorResponse	Validated input, output shape, and the JSON error body
exception / GlobalExceptionHandler	Maps every failure to a JSON error and logs it at WARN
config / DataSeeder	CommandLineRunner that inserts three sample students idempotently

H2 mode: file-based. The datasource URL is `jdbc:h2:file:./data/studentdb;AUTO_SERVER=TRUE;WRITE_DELAY=0`. Data is stored in data/studentdb.mv.db and survives restarts. `AUTO_SERVER=TRUE` lets the H2 console and H2 Shell query the same file while the app runs. `WRITE_DELAY=0` flushes every commit to disk immediately. Without it, H2 buffers commits for about one second, and an early test run lost a record that was written just before the process was force-killed.

3. REST Operations and Validation (Tasks 2 and 3)

Method	Path	Success	Errors
POST	/students	201 + Location	400 validation, 409 duplicate USN
GET	/students	200	none
GET	/students?department=CSE	200 (case-insensitive)	none
GET	/students/{id}	200	404 unknown id, 400 non-numeric id
PUT	/students/{id}	200	400, 404 (never creates), 409
DELETE	/students/{id}	204	404

- **Validation (400):** blank USN, name or department, and a missing semester or one outside 1 to 8, are rejected by Bean Validation before the service runs. The error body lists every failing field.
- **Duplicate USN (409):** checked on create and on update (excluding the record itself). USNs are trimmed and upper-cased, so 1js23cs101 and 1JS23CS101 count as the same USN. A database unique constraint is the final guard.
- **Unknown id (404):** returned by GET, PUT and DELETE. PUT looks up the record first, so it can never create a missing one.
- **No side effects:** every check runs before saving, inside a transaction, so invalid requests never modify stored data. This was verified by re-reading the record after each failed update.
- **Error format:** every error returns JSON with timestamp, status, error, message, path and, for validation failures, fieldErrors.

4. Required Test Evidence

The screenshots in this section were captured with the browser test console bundled with the app at /api-console.html. Each request was sent live to the running JAR, and the panel shows the method, URL, request body, HTTP status, response time, headers and response body. The same requests are in the Postman collection (postman/Student-API.postman_collection.json) and in the README as curl commands.

Note on ids: the created student received id 33 rather than 4 because H2 reserves identity values in blocks of 32. Earlier test runs that force-killed the process discarded the unused block. This is normal database behaviour; ids are only required to be unique, not consecutive.

4.1 Retrieve all students (seeded data)

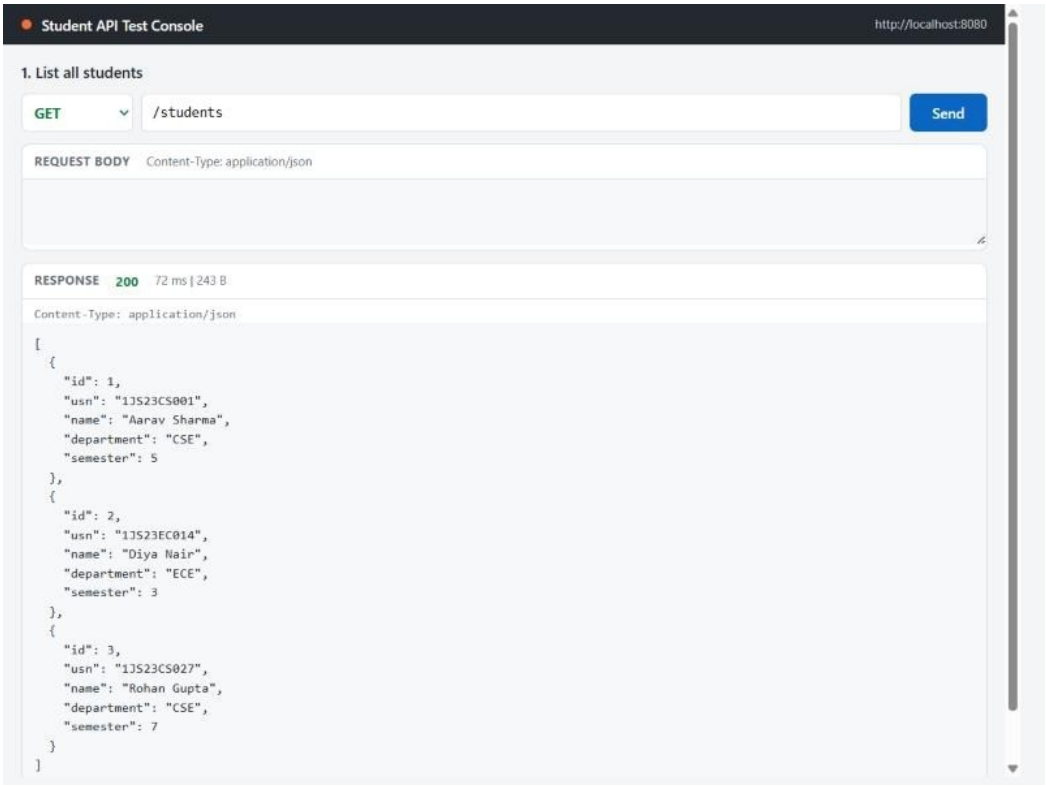


Figure 1. *GET /students returns 200 with the three students inserted by the startup seeder.*

4.2 Valid creation

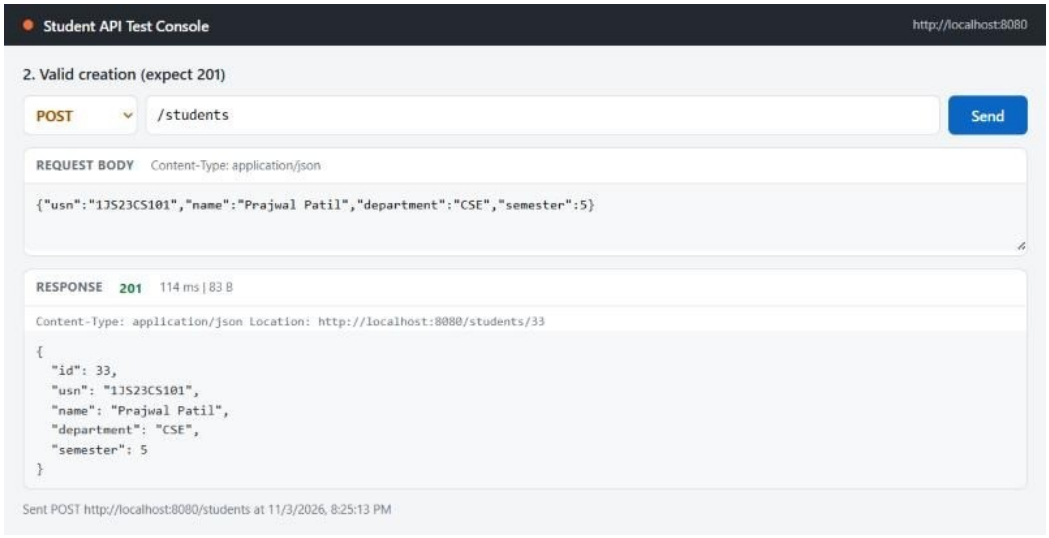


Figure 2. *POST /students returns 201 Created with the generated id and a Location header pointing to /students/33.*

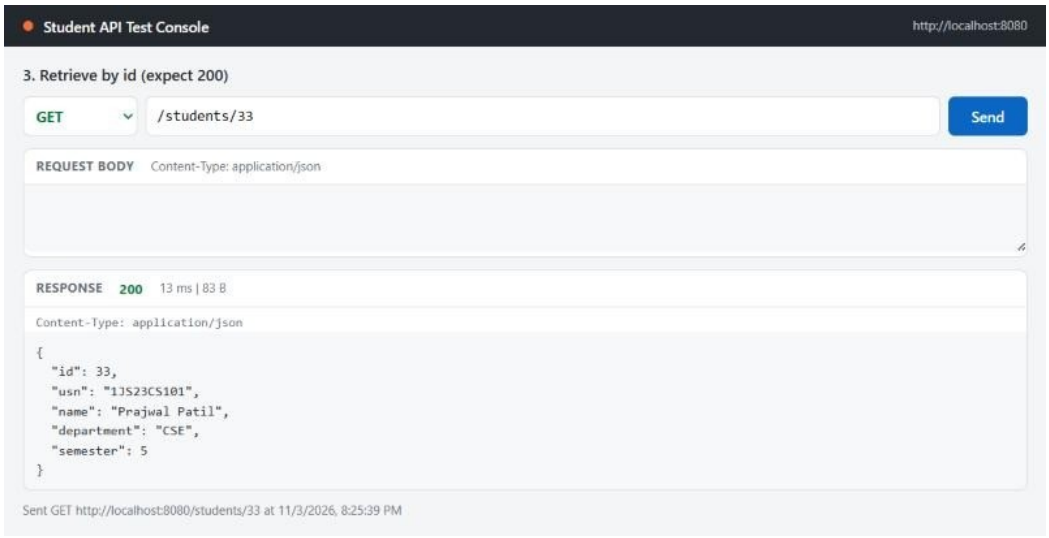


Figure 3. *GET /students/33 retrieves the new record with 200.*

4.3 Duplicate USN

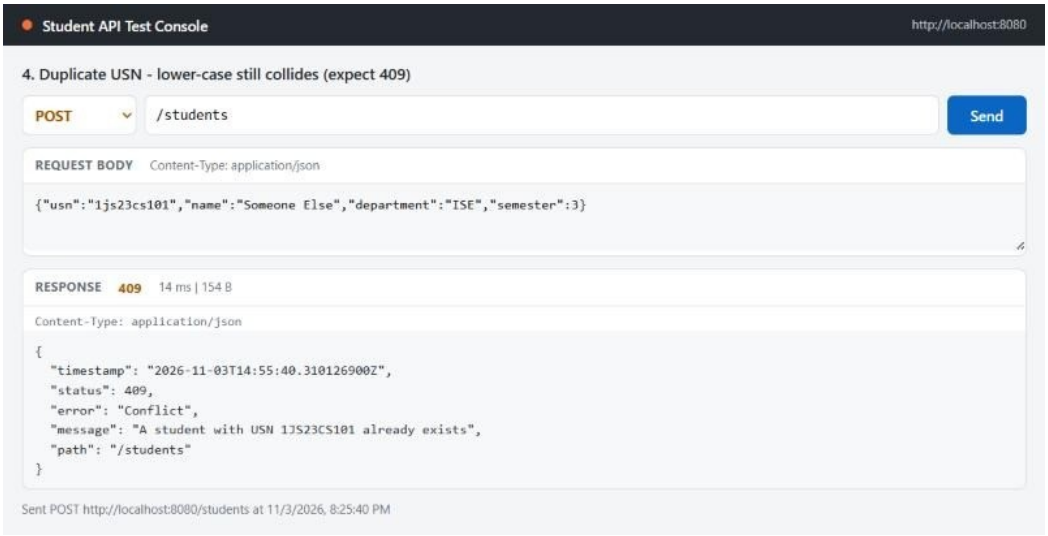


Figure 4. *POST with USN 1js23cs101 (lower case) returns 409 Conflict, because USNs are normalised before the uniqueness check.*

4.4 Invalid semester and blank fields

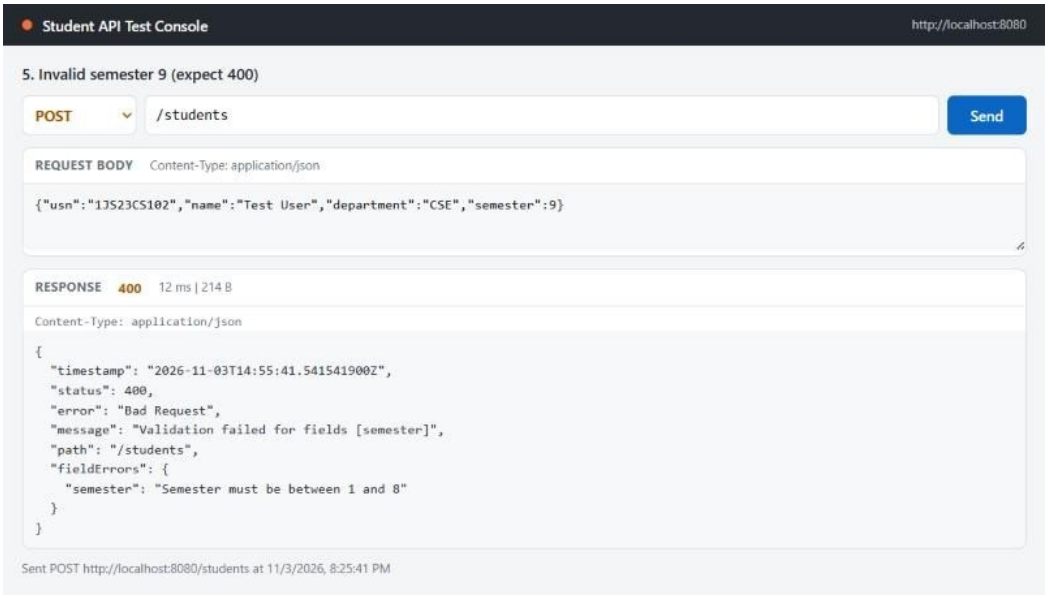


Figure 5. *Semester 9 returns 400 Bad Request with a field-level message.*

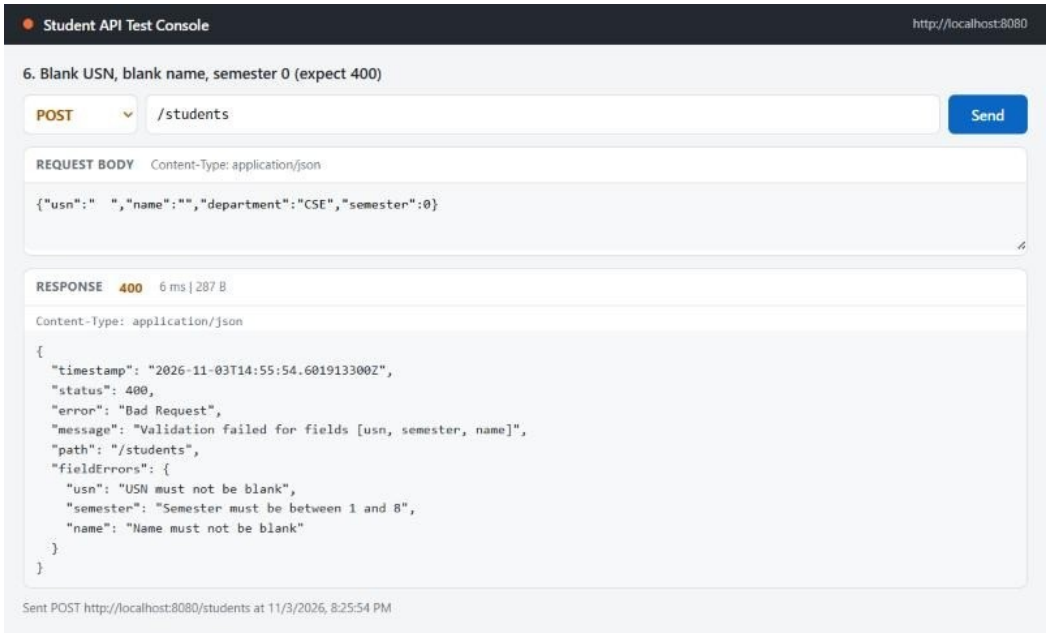


Figure 6. *Blank USN, blank name and semester 0 return 400 with all three field errors reported together.*

4.5 Department filtering

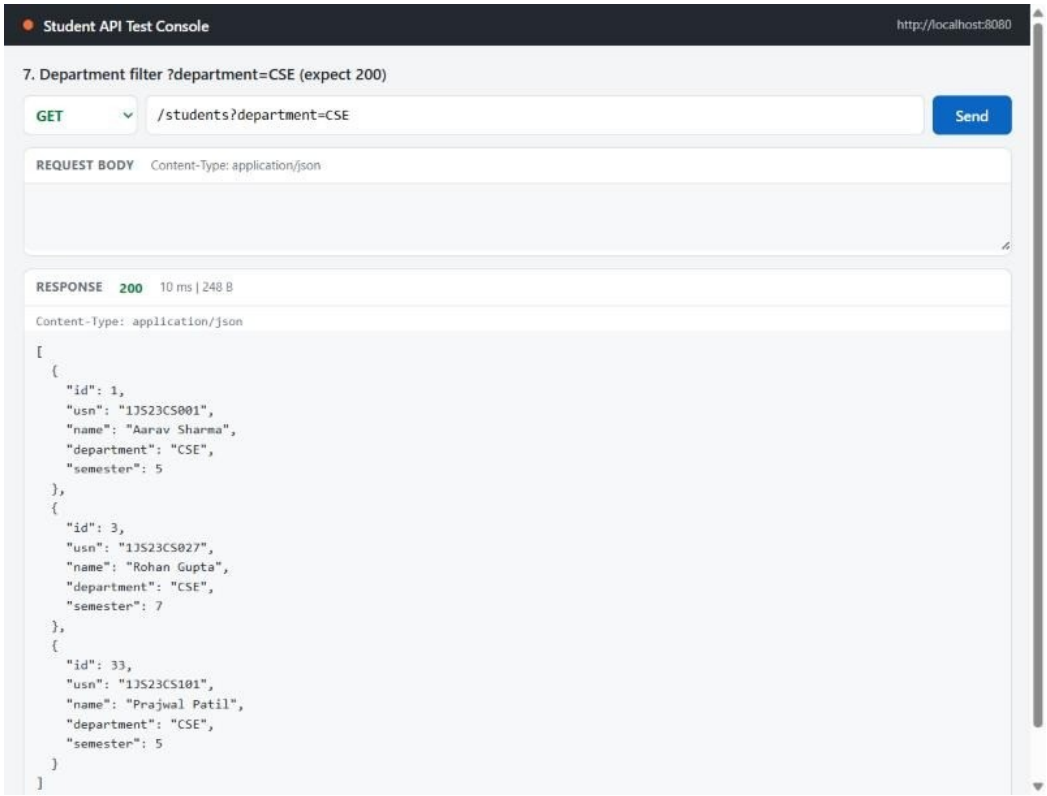


Figure 7. *GET /students?department=CSE returns only CSE students (ECE student id 2 is excluded).*

4.6 Update

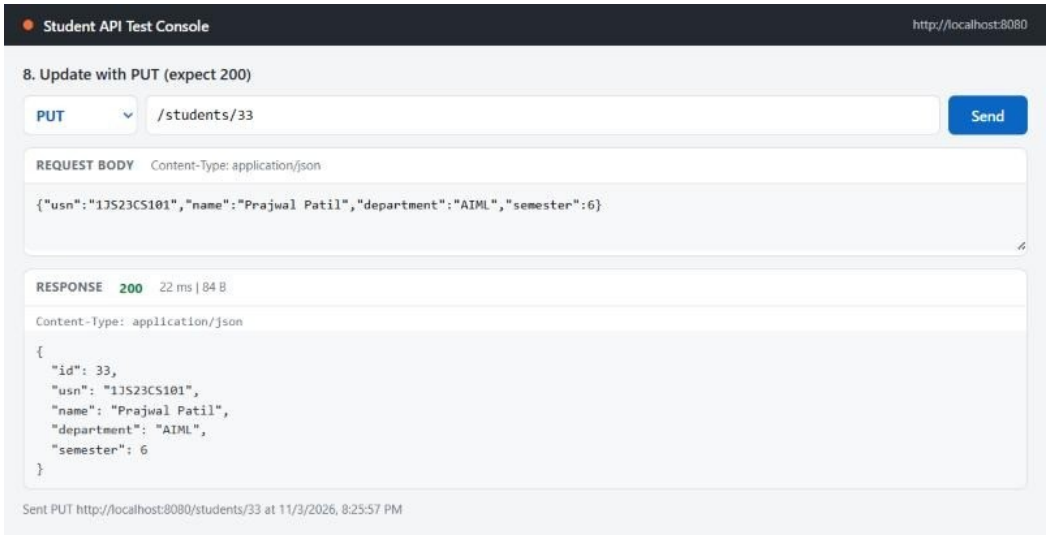


Figure 8. *PUT /students/33 updates department to AIML and semester to 6, returning 200.*

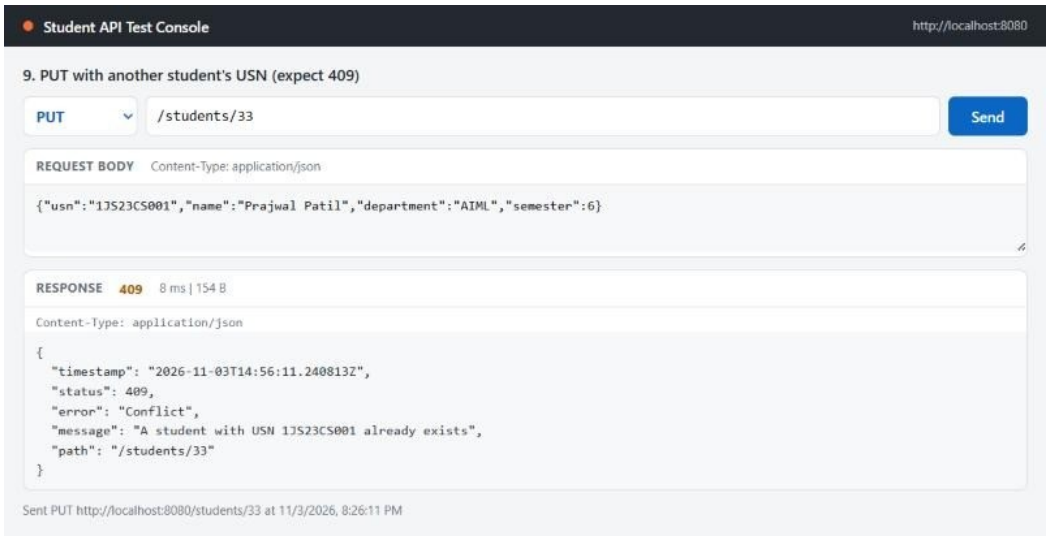


Figure 9. *PUT using a USN that belongs to another student returns 409, and the record is not changed.*

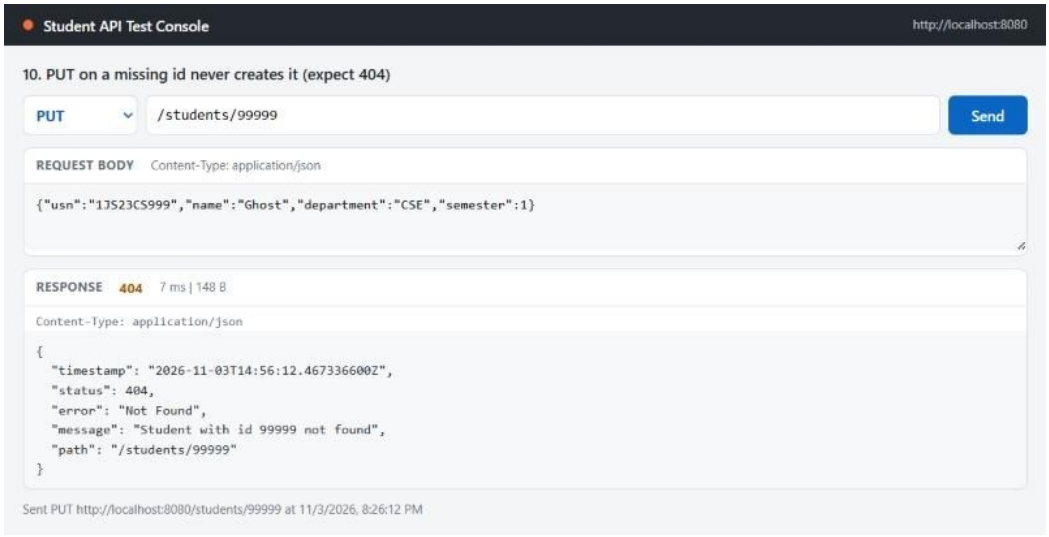


Figure 10. *PUT on a missing id (99999) returns 404: PUT never creates a record.*

4.7 Deletion followed by a 404 lookup



Figure 11. *DELETE /students/33 returns 204 No Content with an empty body.*

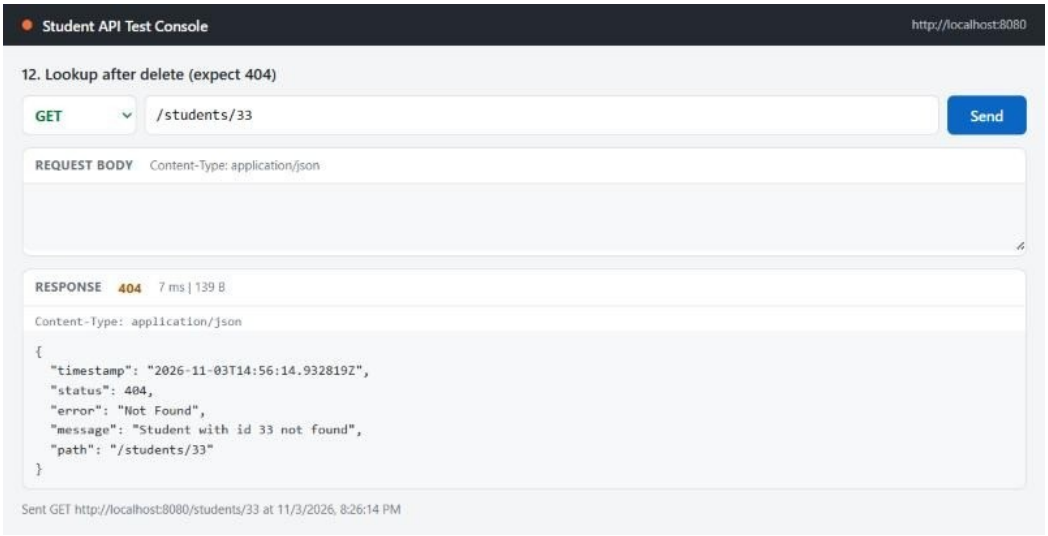


Figure 12. *GET /students/33 after deletion returns 404 with a JSON error message.*

4.8 H2 query result

Queried in the H2 web console at /h2-console against the application database while the app was running. The fourth row is the marker record created in section 6 just before the restart.

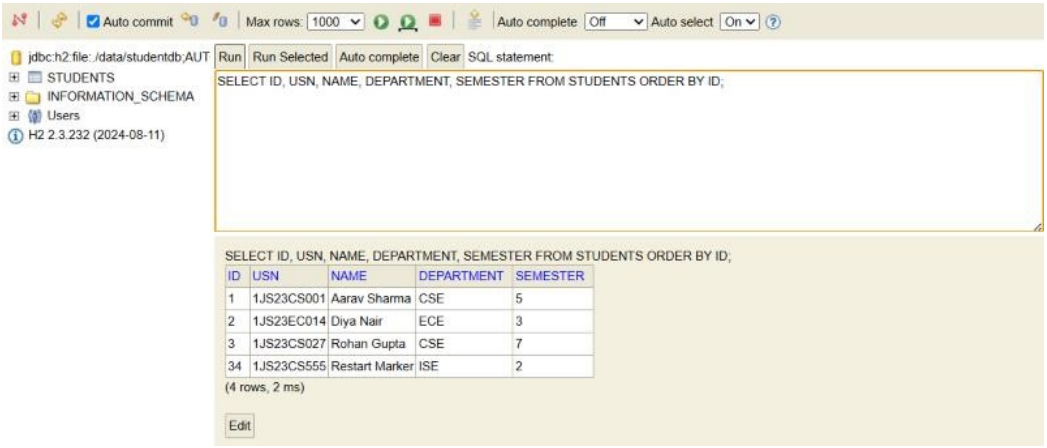


Figure 13. *SELECT ID, USN, NAME, DEPARTMENT, SEMESTER FROM STUDENTS ORDER BY ID in the H2 console (4 rows).*

5. Startup Data and Logging (Task 4)

DataSeeder implements CommandLineRunner and inserts three sample students only when their USN is not already present, so restarts never create duplicate seeds. Logs use the pattern timestamp, level, thread, class name and message, and go to both the console and logs/student-api.log. Startup, every successful create/update/delete (INFO) and every failed request (WARN) are logged, recording only the id, the USN and the failure reason, never a full student record.

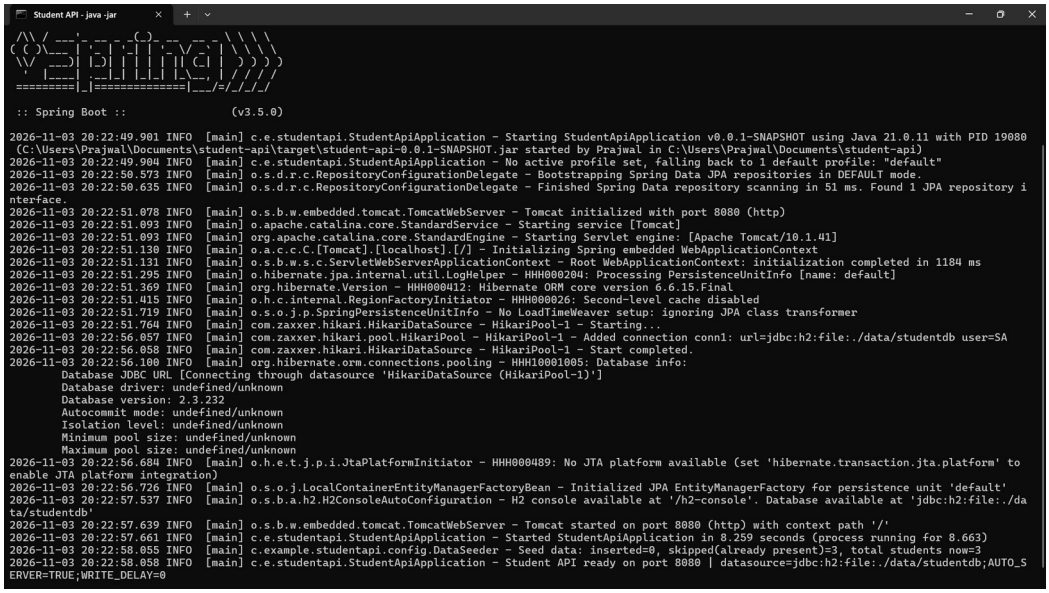


Figure 14. *Startup log of java -jar target/student-api-0.0.1-SNAPSHOT.jar on Spring Boot 3.5.0 / Java 21.*

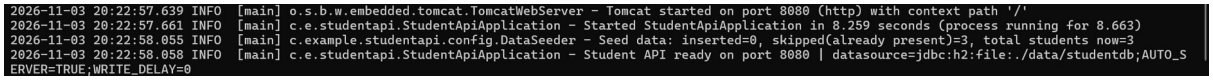


Figure 15. *Detail: Tomcat on port 8080, seeder result, and the "Student API ready" line with the file-based datasource URL.*

Log lines produced by the requests in section 4, copied from logs/student-api.log (thread names shortened from http-nio-8080-exec-N):

```
2026-11-03 20:25:13.795 INFO  [exec-4] c.e.studentapi.service.StudentService - Student created: id=33, usn=1JS23CS101
2026-11-03 20:25:40.310 WARN  [exec-8] c.e.s.exception.GlobalExceptionHandler - Request failed: POST /students -> 409 Conflict: A student with USN 1JS23CS101 already exists
2026-11-03 20:25:41.541 WARN  [exec-10] c.e.s.exception.GlobalExceptionHandler - Request failed: POST /students -> 400 Bad Request: Validation failed for fields [semester]
2026-11-03 20:25:57.276 INFO  [exec-6] c.e.studentapi.service.StudentService - Student updated: id=33, usn=1JS23CS101
2026-11-03 20:26:12.467 WARN  [exec-10] c.e.s.exception.GlobalExceptionHandler - Request failed: PUT /students/99999 -> 404 Not Found: Student with id 99999 not found
2026-11-03 20:26:13.697 INFO  [exec-2] c.e.studentapi.service.StudentService - Student deleted: id=33, usn=1JS23CS101
2026-11-03 20:26:14.931 WARN  [exec-4] c.e.s.exception.GlobalExceptionHandler - Request failed: GET /students/33 -> 404 Not Found: Student with id 33 not found
```

6. Build, Demonstration and Restart (Task 5)

Packaged with `mvn clean package` and run with `java -jar target/student-api-0.0.1-SNAPSHOT.jar`. On Windows, RUN-ME.bat builds the JAR and runs the full create → retrieve →

update → delete sequence, including every error case, the H2 query and a forced-kill restart. It writes all requests and responses to REPORT.md.

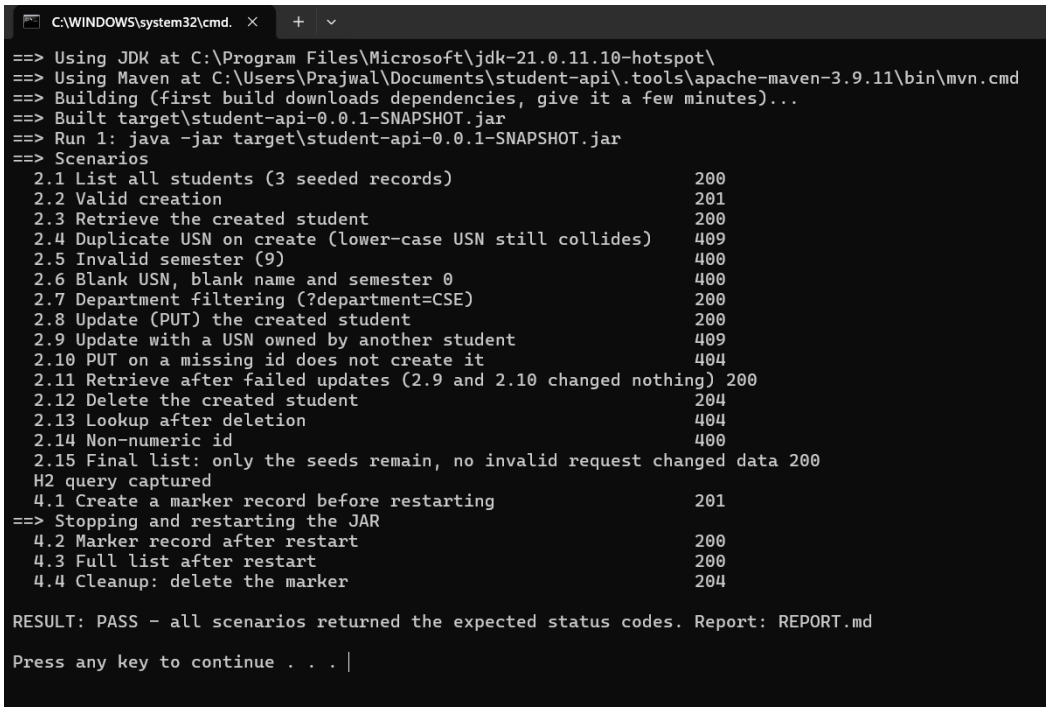


Figure 16. *Automated run on the packaged JAR: all 20 scenarios returned the expected status codes.*

Restart observation

A marker student (USN 1JS23CS555, id 34) was created, the application was stopped, and the same JAR was started again.

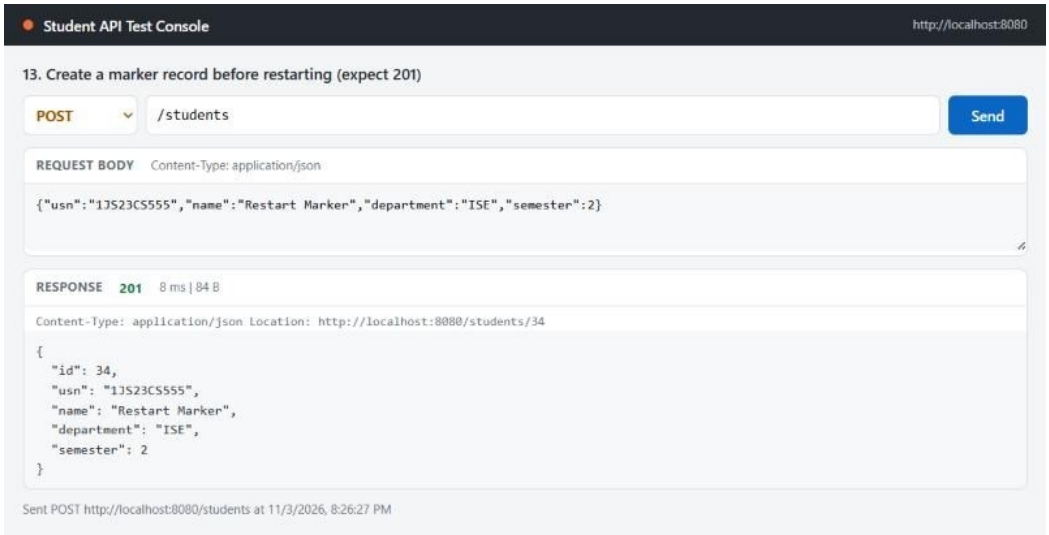


Figure 17. *Before restart: marker record created with 201.*

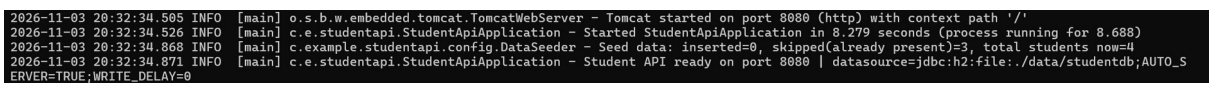


Figure 18. *After restart (new process, PID 27704): the seeder skipped all 3 seeds, and the total is 4, which includes the marker.*

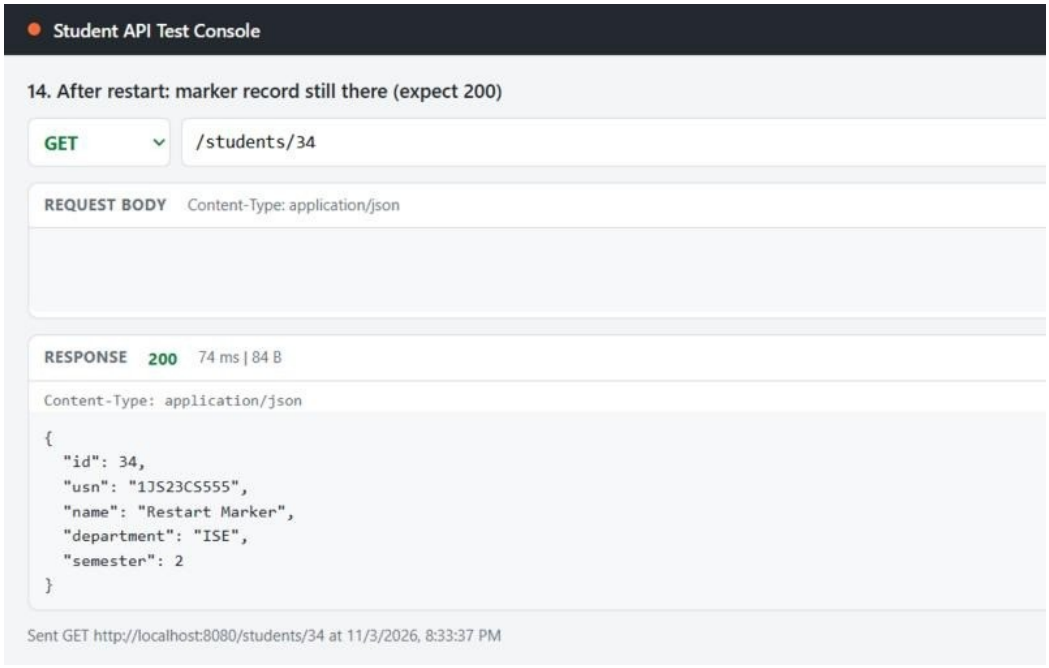


Figure 19. *After restart: GET /students/34 still returns the marker record.*

Observation: because H2 is file-based, every record survives a restart, including data created at runtime. The seeder recognises its three USNs and inserts nothing, so seed records are never duplicated. With an in-memory URL (jdbc:h2:mem:studentdb) the database would be discarded when the JVM exits: only the three seeds would return and the marker would be lost.

7. Why the Service Layer Is Separate from the Controller

The controller only deals with HTTP: it maps URLs, reads JSON, triggers validation and chooses status codes. The service owns the business rules: USN normalisation and uniqueness, "PUT never creates", transactions and mutation logging. Keeping them apart means the same rules apply whether a request arrives over REST, from the startup seeder or from a future batch job, and each layer can be tested on its own (the service with a mocked repository, the controller with a mocked service). It also lets the transport change, for example to GraphQL or messaging, without touching the logic that protects the data.

8. Submission Contents and Commands

File / folder	Purpose
src/, pom.xml, application.properties	Source code, build file and configuration
README.md	Versions, build/run commands, endpoints, sample curl requests
postman/Student-API.postman_collection.json	12 requests with status-code assertions
REPORT.md, evidence/	Generated evidence: every request, status and body, plus console logs
RUN-ME.bat / run-demo.ps1, demo.sh	One-command build + full test run (Windows / Linux and macOS)
START-APP.bat	Build and run the JAR in a visible window

```
mvn clean package
java -jar target/student-api-0.0.1-SNAPSHOT.jar
curl -i -X POST http://localhost:8080/students -H "Content-Type: application/json" -d
'{"usn":"1JS23CS101","name":"Mahantesh R. Patted","department":"CSE","semester":5}'
curl -i "http://localhost:8080/students?department=CSE"
```